C# Machine Problem: Cybersecurity Intrusion Detection System (IDS) & Log Forensics Engine

Problem Scenario

You are stationed as a Security Operations Center (SOC) incident response engineer at a financial technology infrastructure provider. Overnight, automated alarms triggered on the perimeter reverse proxy. The primary web application server was targeted by an adversarial campaign consisting of automated vulnerability scanning, SQL injection exploitation probes, directory path traversals, and distributed credential brute-force attacks.

The Chief Information Security Officer (CISO) requires you to build a standalone, modular, console-based **Intrusion Detection System (IDS) & Forensics Analysis Engine** in C#.

Unlike standard CSV pipeline tasks, this challenge requires you to master **unstructured, real-world server access log files**:

- Extracting dynamic HTTP request tokens (IP, timestamps, methods, URIs, status codes, user agents) from raw text logs.
- Executing **multi-vector signature pattern scanning** to detect SQL Injection (SQLi), Path Traversal (LFI), and malicious exploit scanners.
- Calculating **temporal sliding windows using `DateTime` and `TimeSpan`** to detect credential brute-force attacks (≥ 5 failed logins within 60 seconds).
- Aggregating offending IPs into an active forensic threat matrix with severity classifications.
- Ingesting `server_access.log` and exporting an actionable `security_incident_report.txt` with auto-generated Linux firewall remediation commands.

Learning Competencies Assessed

Competency	Application in Cybersecurity IDS Machine Problem
Log String Tokenization	Isolating IP, timestamp, method, endpoint, status code, and user agent from unstructured log lines using `IndexOf`, `Substring`, and safe parsing.
Temporal Math (`DateTime`)	Parsing timestamps with `DateTime.ParseExact`, tracking request histories, and calculating elapsed `TimeSpan` to detect rapid-fire brute-force spikes.
Signature Pattern Scanning	Multi-pattern string detection (`Contains`, `IndexOf`) for SQLi payloads (`UNION SELECT`, `--`), LFI traversal (`../`, `/etc/passwd`), and scanners (`sqlmap`, `Nikto`).
Collections & State Aggregation	Maintaining synchronized parallel lists (`List<string>`, `List<DateTime>`, `List<int>`) for historical threat tracking and offending IP frequency analysis.
Modular Methods & Tuples	Refactoring detection engines into pure static methods returning multi-value named C# tuples.
Defensive Stream Handling	Safe ingestion and export using `StreamReader` and `StreamWriter` enclosed in `using` statements to guarantee immediate file release.

Namespace Architecture

> **Segregate your project code into two clean namespaces:**
>
> - `SecurityOps.Core`: Houses log tokenizers, attack signature pattern matchers, temporal brute-force analyzers, and risk matrix aggregators.
> - `SecurityOps.IO`: Houses raw log stream ingestion, formatted SOC terminal display tables, and disk export for incident response dossiers.

Required Modular Methods in `SecurityOps.Core.DetectionEngine`

Refactor all detection and forensics logic out of top-level code into standalone, testable static methods:

Method A: Raw Log Entry Tokenizer

- **Identifier:** `ParseLogEntry`
- **Namespace:** `SecurityOps.Core`
- **Signature:** `public static (bool isValid, string ipAddress, DateTime timestamp, string httpMethod, string endpoint, int statusCode, int responseBytes, string userAgent) ParseLogEntry(string rawLogLine)`
- **Log Syntax:** `IP [YYYY-MM-DD HH:mm:ss] "METHOD /endpoint HTTP/1.1" STATUS BYTES "USER_AGENT"`
- **Logic:** Uses string slicing (`IndexOf` , `Substring` , `Split`) to parse tokens. Uses `DateTime.ParseExact(ts, "yyyy-MM-dd HH:mm:ss", null)` and `int.TryParse` for status and bytes. Returns `isValid = false` if malformed, allowing caller to skip cleanly.

Method B: Signature & Attack Vector Scanner

- **Identifier:** `ScanAttackSignatures`
- **Namespace:** `SecurityOps.Core`
- **Signature:** `public static (string threatCategory, int riskScore, string matchedPattern) ScanAttackSignatures(string endpoint, string userAgent)`
- **Threat Signatures & Scores:**
  - **SQL Injection (Score: 85):** Endpoint contains (case-insensitive) `' OR '1'='1` , `UNION SELECT` , `SLEEP(` , `--` , or `INFORMATION_SCHEMA` . Category: `"SQL Injection (SQLi)"` .
  - **Path Traversal / LFI (Score: 90):** Endpoint contains `../` , `..%2f` , `/etc/passwd` , or `win.ini` . Category: `"Path Traversal (LFI)"` .
  - **Exploit Scanners (Score: 70):** User agent contains `sqlmap` , `nikto` , `gobuster` , `nmap` , or `masscan` . Category: `"Automated Exploit Scanner"` .
  - **Benign Traffic (Score: 0):** Category: `"BENIGN TRAFFIC"` , `riskScore = 0` , `matchedPattern = "None"` .

Method C: Temporal Brute-Force Frequency Evaluator

- **Identifier:** `AnalyzeBruteForceActivity`
- **Namespace:** `SecurityOps.Core`
- **Signature:** `public static (bool isBruteForce, int failedAttemptCount) AnalyzeBruteForceActivity(string ipAddress, List<string> ipHistory, List<DateTime> timeHistory, List<string> endpointHistory, List<int> statusHistory, DateTime currentEventTime)`
- **Logic:** Iterates backwards through historical requests matching `ipAddress` . Counts requests to login endpoints (`/login` , `/api/auth` , `/admin/login`) returning HTTP `401` or `403` within a 60-second window (`currentEventTime - pastTime ≤ TimeSpan.FromSeconds(60)`). If `failedAttemptCount ≥ 5` , returns `isBruteForce = true` .

Method D: Comprehensive Threat Severity Classifier

- **Identifier:** `ClassifyIncidentSeverity`
- **Namespace:** `SecurityOps.Core`
- **Signature:** `public static (string severityLabel, string recommendedAction) ClassifyIncidentSeverity(int threatScore, bool isBruteForce, int statusCode)`
- **Decision Hierarchy:**
  - **CRITICAL THREAT:** Threat score ≥ 85, OR `isBruteForce == true` . Action: `"EMERGENCY FIREWALL IP DROP + ISOLATE HOST"` .
  - **HIGH RISK:** Threat score between 65 and 84. Action: `"RATE LIMIT IP + CAPTCHA CHALLENGE"` .
  - **SUSPICIOUS:** Threat score between 30 and 64, or multiple 404/403 errors. Action: `"FLAG FOR MONITORING + ALERT SOC TIER 2"` .
  - **BENIGN:** Threat score < 30. Action: `"PERMIT TRAFFIC"` .

Method E: SOC Threat Matrix & Forensics Aggregator

- **Identifier:** `GenerateThreatMatrix`
- **Namespace:** `SecurityOps.Core`
- **Signature:** `public static (int totalRequests, int criticalCount, int highCount, int suspiciousCount, int benignCount, string topAttackerIp, int maxAttacksBySingleIp) GenerateThreatMatrix(List<string> severities, List<string> ips)`
- **Logic:** Loops through all processed records to aggregate threat counts per severity level. Identifies the primary offending IP address responsible for the highest volume of malicious requests. Guarded against empty datasets.

Interactive SOC Console Menu & Operational Flow

The application entry point must run inside a continuous loop (`do-while` or `while`) presenting the dispatch menu:

```
============================================================
    AEGIS-SOC: AUTOMATED INTRUSION DETECTION SYSTEM (M2)
============================================================
  1. Inspect Single HTTP Log Row (Manual Real-Time Triage)
  2. Ingest Server Access Log (server_access.log)
  3. View Threat Intelligence Dossier & Offending IP Ledger
  4. Export Incident Response Dossier (security_incident_report.txt)
  5. Shutdown IDS Terminal
============================================================
Enter option (1-5):
```

Menu Option Specifications:

- **Option 1 (Manual Real-Time Triage):** Operator pastes or types a raw HTTP access string. Engine extracts tokens, checks attack signatures, determines severity, and displays immediate triage results in the console.
- **Option 2 (Ingest Server Access Log):** Ingests `server_access.log` line by line using `File.ReadAllLines` or `StreamReader`. Parses entries with Method A, checks signatures (Method B), tests temporal brute-force activity (Method C), classifies severity (Method D), and adds records to in-memory audit lists. Skips empty/corrupted lines using `continue`.
- **Option 3 (View Threat Intelligence Dossier):** Prints a formatted tabular ledger of all processed events with color-coded or aligned severity tags, followed by the computed Threat Matrix summary from Method E.
- **Option 4 (Export Incident Response Dossier):** Generates `security_incident_report.txt` containing a summary executive header, itemized attacker logs, and auto-generated Linux firewall remediation commands (`iptables -A INPUT -s <IP> -j DROP`).
- **Option 5 (Shutdown IDS Terminal):** Displays an operational session logout notice and terminates cleanly via `break`.
- **Input Validation:** If the operator submits an out-of-range option or invalid format, display an error message and reprompt the menu without crashing.

Console Display Specifications (Option 3 Sample)

When Option 3 is executed, the application prints a formatted manifest table with aligned columns:

```
================================================================================================================================
                                    AEGIS-SOC INTRUSION DETECTION DOSSIER
================================================================================================================================
Event ID   Timestamp           Source IP        Method  Endpoint / Payload              Risk  Severity Status
--------------------------------------------------------------------------------------------------------------------------------
EVT-5001   2026-10-23 03:14:02  203.0.113.42     GET     /products?id=1' UNION SELECT...    85  CRITICAL THREAT
EVT-5002   2026-10-23 03:14:25  198.51.100.18    POST    /api/login (Failed Attempt #5)     85  CRITICAL THREAT (BRUTE)
EVT-5003   2026-10-23 03:14:30  45.33.32.156     GET     /../../../../etc/passwd            90  CRITICAL THREAT
EVT-5004   2026-10-23 03:14:40  192.168.1.105    GET     /dashboard/profile                  0  BENIGN TRAFFIC
--------------------------------------------------------------------------------------------------------------------------------
THREAT MATRIX SUMMARY:
  Total Events Inspected : 4 events
  Critical Threats Blocked: 3
  High-Risk Probes        : 0
  Suspicious Requests    : 0
  Benign Requests        : 1
  Top Malicious IP       : 198.51.100.18 (5 brute-force auth attempts recorded)
  Firewall Drop Targets  : 3 IP addresses scheduled for immediate iptables blacklisting
================================================================================================================================
```

File Storage & Archival Specifications

1. Input Server Log (`server_access.log`)

Raw Apache / Nginx proxy access format:

```
203.0.113.42 [2026-10-23 03:14:02] "GET /products?id=1' UNION SELECT null,password FROM users-- HTTP/1.1" 200 4520 "Mozilla/5.0"
198.51.100.18 [2026-10-23 03:14:25] "POST /api/login HTTP/1.1" 401 128 "curl/7.68.0"
45.33.32.156 [2026-10-23 03:14:30] "GET /../../../../etc/passwd HTTP/1.1" 404 230 "Nikto/2.1.6"
192.168.1.105 [2026-10-23 03:14:40] "GET /dashboard/profile HTTP/1.1" 200 8920 "Mozilla/5.0"
```

2. Inci
(`secu`

Remed

```
SECUR
Gener
-----
RECOM
iptab
iptab
iptab
-----
SUMMA
```

Verification & Test Scenarios

Event ID	Log Input Attributes	Detected Attack Signature	Classified Severity & Action
EVT-5001	IP: 203.0.113.42 URI: `/products?id=1' UNION SELECT...` Status: 200 | Bytes: 4520	Pattern: `UNION SELECT` Category: SQL Injection (SQLi) Risk Score: 85	**CRITICAL THREAT** *Action: Emergency Firewall IP Drop*
EVT-5002	IP: 198.51.100.18 URI: `/api/login` Status: 401 | Agent: `curl/7.68.0`	5th Failed Login within 60s window Category: Brute-Force Auth Risk Score: 85	**CRITICAL THREAT (BRUTE)** *Action: Emergency Firewall IP Drop*
EVT-5003	IP: 45.33.32.156 URI: `/../../../../etc/passwd` Status: 404 | Agent: `Nikto/2.1.6`	Pattern: `../` and Scanner: `Nikto` Category: Path Traversal (LFI) Risk Score: 90	**CRITICAL THREAT** *Action: Emergency Firewall IP Drop*
EVT-5004	IP: 192.168.1.105 URI: `/dashboard/profile` Status: 200 | Bytes: 8920	No signatures matched Category: Benign Traffic Risk Score: 0	**BENIGN TRAFFIC** *Action: Permit Traffic*

Grading & Verification Checklist

- ☐ **String Tokenization:** Successfully extracts IP, timestamp, method, URI, status code, and user agent from unstructured log syntax without relying on brittle fixed-index offsets.
- ☐ **Temporal Analysis:** Uses `DateTime.ParseExact` and `TimeSpan` math to accurately identify rapid-fire brute-force threshold breaches within rolling 60-second windows.
- ☐ **Pattern Recognition:** Scans for SQLi, directory traversal, and exploit scanners using case-insensitive substring search methods (`Contains` , `IndexOf`).
- ☐ **Tuple Return Values:** Modular methods return strongly typed multi-value C# tuples for parsing, attack classification, and batch threat matrices.
- ☐ **Loop & Error Guarding:** Interactive loop handles invalid input; file reader skips blank lines with `continue` ; division calculations guard against zero denominators.
- ☐ **Deterministic Resource Management:** File streams are wrapped in `using` blocks to guarantee file handles release immediately upon completion.
- ☐ **Zero Warnings / Zero Errors:** Application compiles cleanly under .NET 8 / 9 with 0 errors and 0 warnings.